

Introduction

This report covers the implementation details and answers the questions for hw1 of the deep learning course.

Implementation Details

- Sigmoid

```
def sigmoid(x):  
    return 1.0 / (1.0 + np.exp(-x))  
  
def d_sigmoid(sigmoid_x):  
    return sigmoid_x * (1.0 - sigmoid_x)
```

Copy and paste. The sigmoid_x values can be accessed from the store activated layers.

- NN architecture

```
nn = NN(X, y, [2, 4, 4, 1], model_name="model_linear.pkl", activation=activation, optim="adam")
```

The neural network has 2 hidden layers with 4 neurons each, and by default all layers use the sigmoid activation function.

- Backpropagation

```
def backward(self, lr):  
    dLdA = (self.activated[-1] - self.y) / self.num_samples  
  
    d_weights = [None] * self.layer_count  
    d_biases = [None] * self.layer_count  
  
    for i in reversed(range(self.layer_count - 1)):  
        if self.activation == "sigmoid":  
            dLdZ = dLdA * d_sigmoid(self.activated[i])  
        elif self.activation == "relu":  
            dLdZ = dLdA * d_relu(self.layers[i])  
        else:  
            raise Exception("Invalid activation function setting")  
  
        d_weights[i] = (self.activated[i - 1].T.dot(dLdZ)) if i > 0 else (self.X.T.dot(dLdZ))  
        d_biases[i] = np.sum(dLdZ, axis=0)  
  
        if i > 0:  
            dLdA = dLdZ.dot(self.weights[i].T)  
  
    if self.optim == "adam":  
        self.t += 1  
        for i in range(self.layer_count - 1):  
            self.m_weights[i] = self.beta1 * self.m_weights[i] + (1 - self.beta1) * d_weights[i]  
            self.m_biases[i] = self.beta1 * self.m_biases[i] + (1 - self.beta1) * d_biases[i]  
  
            self.v_weights[i] = self.beta2 * self.v_weights[i] + (1 - self.beta2) * (d_weights[i] ** 2)  
            self.v_biases[i] = self.beta2 * self.v_biases[i] + (1 - self.beta2) * (d_biases[i] ** 2)  
  
            m_hat_w = self.m_weights[i] / (1 - self.beta1 ** self.t)  
            v_hat_w = self.v_weights[i] / (1 - self.beta2 ** self.t)  
  
            m_hat_b = self.m_biases[i] / (1 - self.beta1 ** self.t)  
            v_hat_b = self.v_biases[i] / (1 - self.beta2 ** self.t)  
  
            self.weights[i] -= lr * m_hat_w / (np.sqrt(v_hat_w) + self.epsilon)  
            self.biases[i] -= lr * m_hat_b / (np.sqrt(v_hat_b) + self.epsilon)  
    elif self.optim == "sgd":  
        for i in range(self.layer_count - 1):  
            self.weights[i] -= lr * d_weights[i]  
            self.biases[i] -= lr * d_biases[i]  
    else:  
        raise Exception("Invalid optimizer setting")
```

MSE is chosen as the loss for its simplicity.

$d\text{Loss}/d\text{Activated}$ (dL/dA) is calculated directly using the derivative of MSE (the constant 2 is omitted here since it shouldn't effect the outcome)

Then, iterate through all layers in reverse order and calculate $d\text{Loss}/d\text{Layer}$ (dL/dZ) using the chain rule ($dL/dA * dA/dZ$, the latter term is just the derivative of the activation function)

To find dL/dW , again, using the chain rule, dZ/dW is needed. We know that because $Z = A_{\text{prev}} * W$, dZ/dW is just A_{prev} , so dL/dW can be written as $dL/dZ * A_{\text{prev}}$, except for the input layer where an activation function is not applied, X is used instead of A_{prev} .

For $dBias$, dZ/dB is 1 since adding B shifts Z by B , hence dL/dB is equivalent to dL/dZ . And because the implementation uses batch gradient descent, the gradient across the entire batch needs to be summed up.

At the end of each iteration, dL/dA is moved up a layer with $dL/dA_{\text{prev}} = dL/dZ * dZ/dA_{\text{prev}}$. Again, from $Z = A_{\text{prev}} * W$, $dZ/dA_{\text{prev}} = W$, $dL/dA_{\text{prev}} = dL/dZ * W$.

After backpropagation, all deltas are stored in `d_weights` and `d_biases`. In the case of simple SGD, these arrays are multiplied by the learning rate and deducted from the actual weights and biases.

- Note: to run the script
Run

```
python hw1.py --config {linear, xor} --mode train
```

To train the model with default settings and produce a pkl file that stores the model (`model_linear` or `model_xor`). Any pre-existing model file will be overwritten.

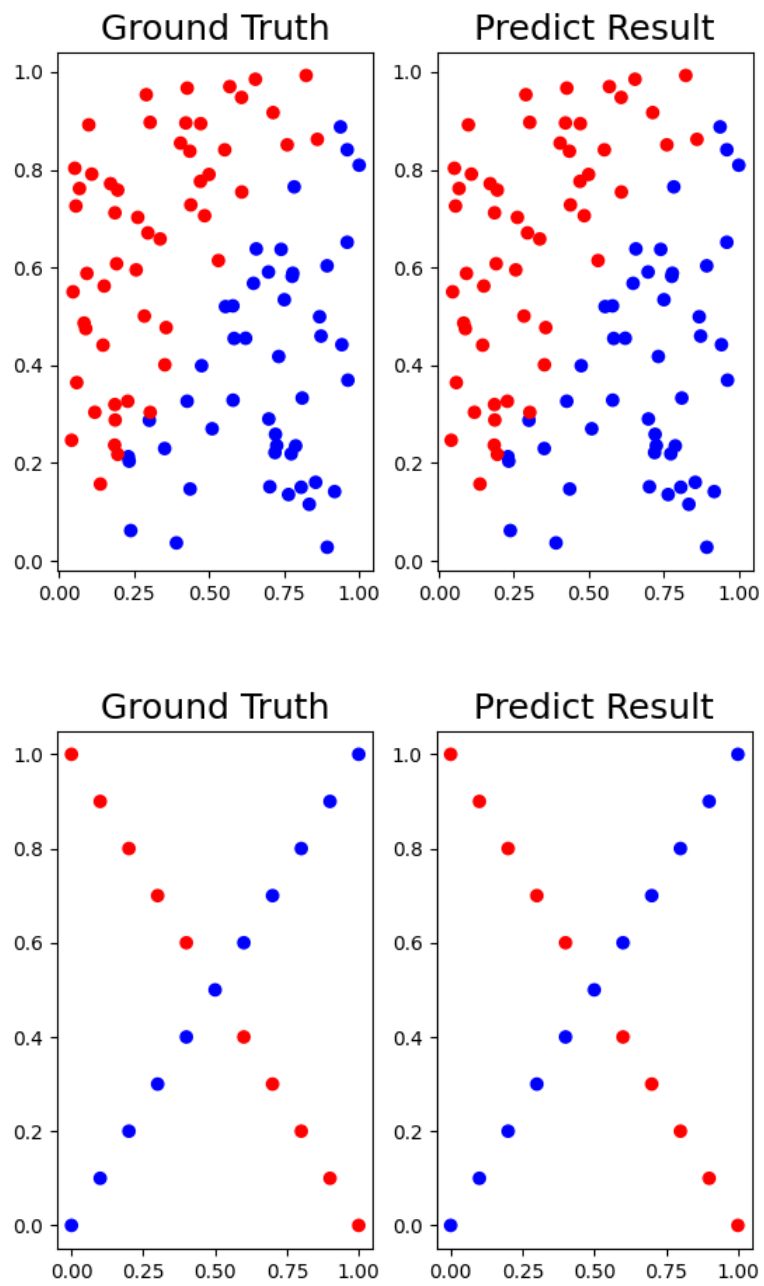
Then run

```
python hw1.py --config {linear, xor} --mode test
```

To load and test the model that is stored at the same directory of the python file.

Experiment Results

- Scatter plot



- Accuracy

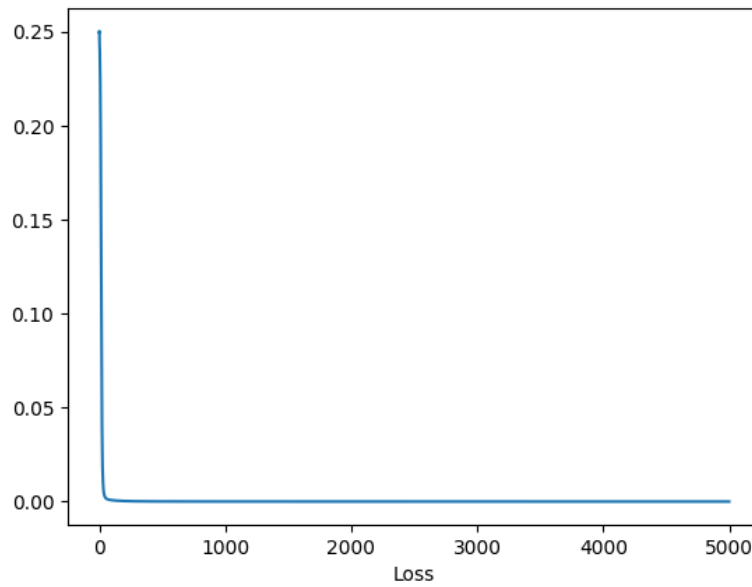
Linear

```
iter 79 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 80 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 81 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 82 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 83 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 84 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 85 |      Ground Truth: 1 |      prediction: 1 (0.992981) |
iter 86 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 87 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 88 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 89 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 90 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 91 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 92 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 93 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 94 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 95 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 96 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 97 |      Ground Truth: 0 |      prediction: 0 (0.000000) |
iter 98 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 99 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
iter 100 |      Ground Truth: 1 |      prediction: 1 (1.000000) |
prediction loss: 0.0014928032837365382
prediction accuracy: 1.0
```

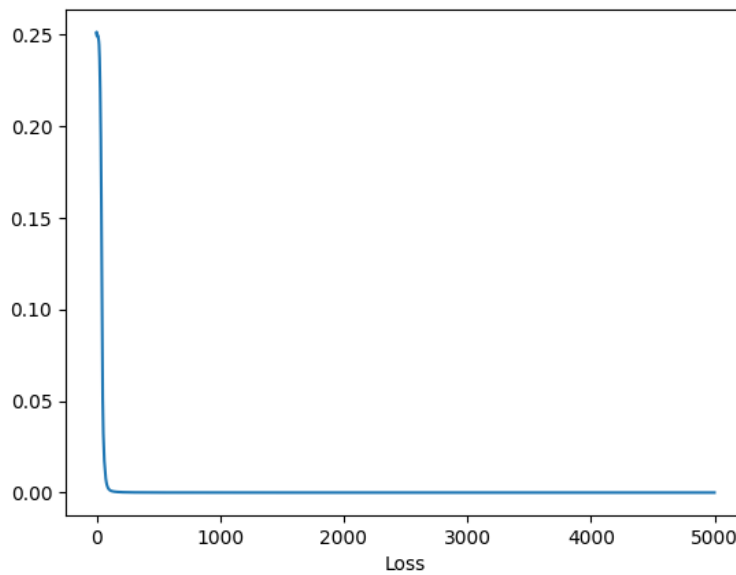
XOR

```
Mode: test
Config: xor
iter 1 |      Ground Truth: 0 |      prediction: 0 (0.000179) |
iter 2 |      Ground Truth: 1 |      prediction: 1 (0.999999) |
iter 3 |      Ground Truth: 0 |      prediction: 0 (0.000010) |
iter 4 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
iter 5 |      Ground Truth: 0 |      prediction: 0 (0.000003) |
iter 6 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
iter 7 |      Ground Truth: 0 |      prediction: 0 (0.000006) |
iter 8 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
iter 9 |      Ground Truth: 0 |      prediction: 0 (0.000133) |
iter 10 |      Ground Truth: 1 |      prediction: 1 (0.998863) |
iter 11 |      Ground Truth: 0 |      prediction: 0 (0.001437) |
iter 12 |      Ground Truth: 0 |      prediction: 0 (0.000586) |
iter 13 |      Ground Truth: 1 |      prediction: 1 (0.998917) |
iter 14 |      Ground Truth: 0 |      prediction: 0 (0.000109) |
iter 15 |      Ground Truth: 1 |      prediction: 1 (0.999997) |
iter 16 |      Ground Truth: 0 |      prediction: 0 (0.000029) |
iter 17 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
iter 18 |      Ground Truth: 0 |      prediction: 0 (0.000012) |
iter 19 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
iter 20 |      Ground Truth: 0 |      prediction: 0 (0.000007) |
iter 21 |      Ground Truth: 1 |      prediction: 1 (0.999998) |
prediction loss: 2.3525551513621652e-07
prediction accuracy: 1.0
```

- Learning Curve
Linear



XOR



- Miscellaneous

From my testing, from time to time a model using relu exclusively as its activation function will get stuck at a certain loss value.

Discussion

- Different Learning Rates

For SGD, if the learning rate is too large, the final accuracy will be suboptimal, and if the learning rate is too small, the model won't be able to converge. This is why I added a `lr_scheduling` parameter that decays the learning rate every 20% progress, although the adam optimizer dynamically adjusts the learning rate in a more optimal way, making the difference not noticeable.

- Different number of hidden units.

A network with no hidden layers was able to learn the Linear distribution with over 99% accuracy, while the minimum config that was able to learn the xor distribution was [2, 2, 1] though with certain initial weights it was only able to reach 95% training accuracy.

- Without activation functions

Without activation functions the model successfully learned the linear separation distribution but not the XOR distribution.

Questions

- What is the purpose of activation functions?

Activation functions introduce non-linearity to the network which enables the model to approximate complex and arbitrary functions.

- What might happen if the learning rate is too large or too small?

Quote discussion:

if the learning rate is too large, the final accuracy will be suboptimal, and if the learning rate is too small, the model won't be able to converge.

- What is the purpose of weights and biases in a neural network?

Weights determine the strength of connections between neurons, while biases allow for fine-tuning and adjusting predictions.

Extra

- Implement different optimizers

In addition to SGD, I've implemented the adam optimizer this can be changed by modifying the `optim` parameter of the NN class. The code is just a "transcript" of the algorithms shown [here](#).

```
if self.optim == "adam":
    self.t += 1
    for i in range(self.layer_count - 1):
        self.m_weights[i] = self.beta1 * self.m_weights[i] + (1 - self.beta1) * d_weights[i]
        self.m_biases[i] = self.beta1 * self.m_biases[i] + (1 - self.beta1) * d_biases[i]

        self.v_weights[i] = self.beta2 * self.v_weights[i] + (1 - self.beta2) * (d_weights[i] ** 2)
        self.v_biases[i] = self.beta2 * self.v_biases[i] + (1 - self.beta2) * (d_biases[i] ** 2)

        m_hat_w = self.m_weights[i] / (1 - self.beta1 ** self.t)
        v_hat_w = self.v_weights[i] / (1 - self.beta2 ** self.t)

        m_hat_b = self.m_biases[i] / (1 - self.beta1 ** self.t)
        v_hat_b = self.v_biases[i] / (1 - self.beta2 ** self.t)

        self.weights[i] -= lr * m_hat_w / (np.sqrt(v_hat_w) + self.epsilon)
        self.biases[i] -= lr * m_hat_b / (np.sqrt(v_hat_b) + self.epsilon)
```

- Implement different activation functions

I've implemented the relu activation function, which can be set by passing in the "`-activation relu`" argument.